Architektury i technologie systemów internetowych – dokumentacja techniczna „Cyfrowe archiwum społecznościowe”

Jan Żmuda 311640

Spis treści

Rozdział 1. Zakres projektu....3

Rozdział 2. Projektowanie systemu....3

Rozdział 3. Architektura systemu....4

Warstwa Logiki i Danych (Backend)....4

Warstwa Prezentacji (Frontend)....7

Rozdział 4. Bibliografia i użyte strony....8

Rozdział 1. Zakres projektu

W ramach projektu należało zrealizować aplikację internetową do obsługi cyfrowego archiwum społecznościowego. System przewiduje następujące role:

- Administrator
- Przeglądający
- Twórca

oraz spełnia wymagania:
- Administrator może usuwać zdjęcia, edytować ich opis i metadane oraz zablokować Twórcę, by nie mógł dodawać nowych zdjęć.

- Przeglądający może przeglądać i wyszukiwać za pomocą narzędzi zdjęcia dostępne w archiwum. Aplikacja oferuje opcje wyszukiwania po lokalizacji, zawieranej frazie czy hierarchicznej strukturze.

- Twórca ma wszystkie możliwości Przeglądającego oraz do tego odpowiada za zarządzanie swoimi materiałami oraz ich opisem. Loguje się do serwisu za pomocą konta Google, może m.in. przesyłać, modyfikować i usuwać swoje zdjęcia.

- System udostępnia publiczne API Rest umożliwiające wyszukiwanie materiałów oraz prywatne API umożliwiające dodawanie materiałów przez Twórcę.

- Interfejs został zaprojektowany z myślą o pełnej dostępności cyfrowej dla osób z niepełnosprawnościami, spełnia on wymogi WCAG 2.1 AA, co potwierdzono testem za pomocą narzędzia WAVE.

Rozdział 2. Projektowanie systemu

Na podstawie wymagań postanowiono wykonać aplikację „Cyfrowe Archiwum Społecznościowe”, korzystając z języka programowania Python, lekkiego frameworka FastAPI oraz bazy danych SQLite po stronie backendu, a także narzędzia Svelte z wykorzystaniem stylów Bootstrap 5. Jako kompilator kodu frontendowego wykorzystano środowisko Vite.

Powodem użycia tych technologii była ich prostota przy jednoczesnym dostarczaniu wszystkich funkcji niezbędnych do spełnienia wymagań projektu. Dodatkowo wybrane narzędzia charakteryzują się szybkością i lekkością w działaniu, co bezpośrednio przekłada się na responsywność interfejsu klienta. Z kolei biblioteka Bootstrap 5 pozwoliła na sprawną implementację odpowiednich motywów graficznych – jasnego, ciemnego oraz trybu wysokiego kontrastu, dedykowanego osobom słabowidzącym.” Wybrano bazę SQLite ze względu na prostotę i szybkość w implementacji. Choć istnieją bazy danych z lepszymi funkcjami wyszukiwania przestrzennego (np. PostgreSQL z wtyczką PostGIS), tak wymagają one większej konfiguracji ze strony backendu i niepotrzebnie komplikują

projekt tego rozmiaru. Sama baza przechowuje jedynie metadane zdjęć oraz ścieżkę do zdjęcia zapisanego fizycznie na dysku serwera backendu (jest to efektywniejsze dla bazy danych niż umieszczanie w niej zdjęć w formacie dużych obiektów binarnych).

Rozdział 3. Architektura systemu

System został zbudowany w schemacie klient-serwer i dzieli się logicznie na dwie oddzielone niezależne aplikacje: Backend (API i Baza Danych) oraz Frontend (Wygląd i logika przeglądarkowa klienta). Aplikacja została stworzona jako SPA (Single Page Application). Taki model pozwala na znaczące zmniejszenie liczby zapytań do serwera i uniknięcie ciągłego przeładowywania interfejsu przeglądarki po każdym kliknięciu.

Warstwa Logiki i Danych (Backend)

Rdzeniem warstwy serwerowej aplikacji jest API REST, zbudowane w języku Python przy użyciu zoptymalizowanego frameworka FastAPI. Komunikacja między aplikacją a bazą danych jest obsługiwana przez bibliotekę SQLAlchemy. Wszystkie wygenerowane relacje i tabele zapisywane są bezpiecznie w lokalnym, plikowym formacie silnika SQLite.

Architektura backendu została podzielona na dedykowane pliki zadaniowe:

main.py - Główny plik wejściowy. Służy do inicjalizacji całej instancji aplikacji serwerowej za pomocą platformy Uvicorn, wczytywania zmiennych konfiguracyjnych (CORS) powiązanych ze środowiskiem oraz udostępniania na żywo wirtualnego folderu /uploads ze zdjęciami dostępnymi w serwisie. Przede wszystkim zawiera on również klamry definiujące wszystkie endpointy dostępu dla klientów, odpowiedzialne za operacje na materiałach (CRUD dla galerii) oraz operacje odpowiedzialne za uwierzytelnionych Twórców.

database.py - Moduł konfiguracyjny, realizujący połączenia między warstwą Pythona a fizycznym plikiem bazy “archiwum.db”. Odpowiada za bezpieczne otwieranie i zamykanie połączeń z plikiem bazy danych dla każdego zapytania użytkownika. Zarządza ruchem tak, aby w momencie jednoczesnego dodawania zdjęć przez wielu Twórców nie doszło do zablokowania lub uszkodzenia pliku SQLite. Dodatkowo udostępnia podstawowe ustawienia, z których korzystają następnie definicje tabel w pliku models.py

models.py - Plik pełniący funkcję szablonu dla bazy danych projektu. Tłumaczy on kod z języka Python na instrukcje zrozumiałe dla bazy SQLite, bez konieczności pisania ręcznych zapytań SQL. To w nim zdefiniowano wygląd dwóch głównych tabel w systemie:

Tabela User - przechowująca dane użytkowników, identyfikatory pobrane z Google oraz ich rolę w systemie, np. "Administrator".

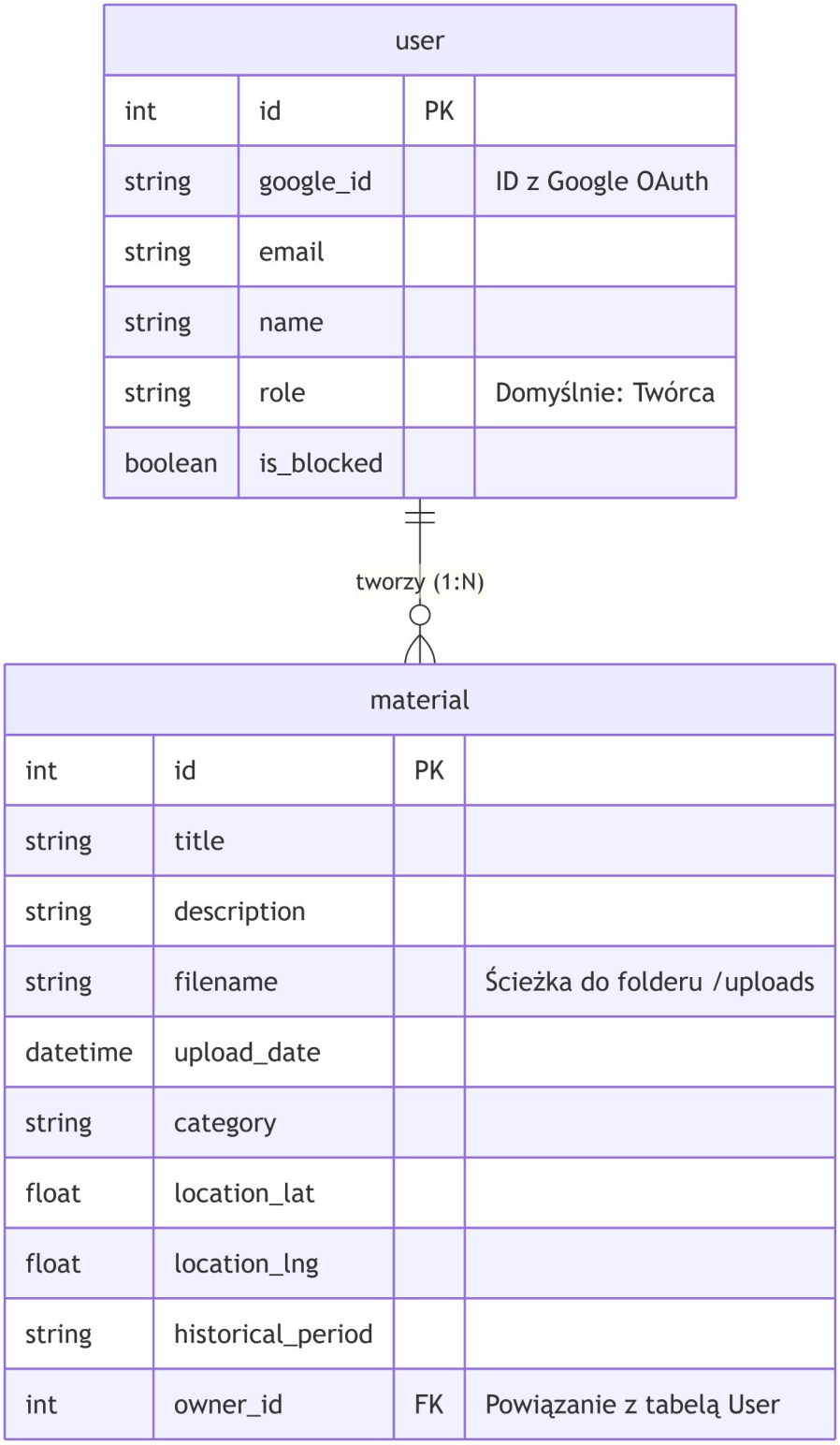
Tabela Material - przechowująca wszystkie wpisy i zdjęcia.

schemas.py - To zbiór klas (tzw. schematów), które sprawdzają poprawność przesyłanych informacji. Korzystając z biblioteki Pydantic, upewnia się on, że np. nadesłany formularz

zawiera wszystkie wymagane pola we właściwym formacie (np. czy email ma znak małpy, a tytuł jest tekstem). Zapobiega to błędom aplikacji i wpuszczeniu do bazy niekompletnych informacji.

auth.py - Plik odpowiedzialny wyłącznie za bezpieczeństwo i logowanie. Po zalogowaniu przez Google, system tworzy dla użytkownika token JWT. W tym module zawarta jest też funkcja sprawdzająca poprawność takiego tokenu przy próbie dodania lub usunięcia materiału z archiwum.

Struktura relacyjna bazy danych (ERD):



Warstwa Prezentacji (Frontend)

Frontend systemu został zrealizowany jako aplikacja jednostronicowa, przekazując całą logikę renderowania interfejsu (HTML/CSS) do przeglądarki użytkownika. Pozwoliło to na minimalizację opóźnień sieciowych oraz odciążenie serwera API. Do kompilacji wybrano Svelte ze środowiskiem developerskim Vite.

Struktura katalogowa frontendu korzysta ze standardowego, bazowego szablonu frameworka, stworzonego przez menedżer Vite. Cała logika biznesowa projektu znajduje się w katalogu frontend/src/. Wygenerowane przez środowisko startowe pliki konfiguracyjne, takie jak vite.config.js czy svelte.config.js, zostały pozostawione nietknięte.

Struktura interfejsu klienta opiera się o hermetyczne moduły nazywane komponentami (pliki z rozszerzeniem .svelte):

App.svelte – Centralny komponent, który pełni rolę korzenia całej aplikacji. Znajdują się w nim niezmienne elementy strony: pasek nawigacyjny i przycisk do logowania za pomocą konta Google. Ten plik nasłuchuje globalnych zmiennych stanu i na tej podstawie decyduje, czy wyświetlić w danej chwili galerię, czy też panel twórcy.

store.js – Pełni funkcję globalnej pamięci (tzw. Svelte Store). Przechowuje m.in. token JWS zalogowanego użytkownika, jego adres e-mail i rolę.

Wyszukiwarka.svelte - Część interfejsu odpowiedzialna za filtrowanie zdjęć. Wyświetla formularz, w którym Przeglądający może wpisać szukane słowo, wybrać na liście kategorię zdjęcia czy też zaznaczyć okres historyczny. Gdy użytkownik zmienia te pola, komponent na bieżąco przekazuje te informacje wyższemu komponentowi, by aktywować wybrane filtry. Komunikacja odbywa się za pomocą funkcji callback.

Galeria.svelte - Główny widok ze zdjęciami dostępny dla każdego Przeglądającego. To tutaj strona łączy się z bazą danych, prosząc o listę zdjęć. Komponent ten odbiera zaznaczone opcje ustawione w Wyszukiwarka.svelte oraz na mapie i odsyła do backendu prośbę tylko o te zdjęcia, które pasują do aktualnych kryteriów. Wyświetla fotografie w formie siatki wraz z ich opisami

Mapa.svelte - Moduł używający biblioteki Leaflet. Służy do wyświetlenia pinesek reprezentujące zdjęcia w bazie na mapie. Jednocześnie w galerii wyświetlone są tylko zdjęcia znajdujące się w obszarze widocznym na mapie, co pozwala szybko wyszukiwać materiały po lokalizacji.

PanelZarządzania.svelte – Interaktywny interfejs, niedostępny dla niezalogowanych użytkowników. Dostęp do niego jest otwierany dopiero po zalogowaniu jako "Twórca" lub "Administrator". Odpowiada za:

- Dodawanie zdjęć. Realizuje to za pomocą formularza dostępnego dla użytkownika.

- Przeglądanie i edycja zdjęć. Wyświetla tabelę z dodanymi przez Twórcę zdjęciami lub wszystkie zdjęcia w serwisie, jeśli otwiera ten panel Administrator. Z tego widoku można usuwać lub edytować zdjęcia. Dodatkowo administrator ma opcję zablokowania

użytkownika będącego autorem wybranego zdjęcia, uniemożliwiając mu dalsze wstawianie zdjęć.

MotywWCAG.svelte – Menadżer stylów wizualnych. Za pomocą klas biblioteki Bootstrap 5 wybiera preferowany kontrast i ubarwienie elementów strony.

index.css – Arkusz stylów, ma zastosowanie dla wszystkich elementów strony. Posłużył do zdefiniowania stylów elementów strony w trybie wysokiego kontrastu (Bootstrap 5 nie posiada domyślnie takiego stylu).

Rozdział 4. Bibliografia i użyte strony

Dokumentacja Python - https://docs.python.org/3/

Dokumentacja frameworka FastAPI - https://fastapi.tiangolo.com/

Podrecznik platformy SQLite - https://www.sqlite.org/docs.html

Pomoc dla komponentów Svelte - https://svelte.dev/docs/svelte

Zasoby platformy Bootstrap 5 w tworzeniu UI - https://getbootstrap.com/docs/5.3/getting-started/introduction/

Dokumentacja Google Identity Services - https://developers.google.com/identity

Repozytorium interaktywnych map JS Leaflet - https://leafletjs.com/reference.html

Strona za pomocą której wygenerowano diagram bazy danych - https://www.mermaidonline.live/mermaid-to-image

Wtyczki WAVE ze strony WebAIM - https://wave.webaim.org/